

Nexus — LinkedIn Redesign

Integration Test Specification

CS485 AI-Assisted Software Engineering — Spring 2026

Team: [Krishi Shah](#), [Dhyani Soni](#), [Saanvi Elaty](#), [Victor Jimenez](#)

This document defines the English-language integration test specification for the Nexus application. Integration tests verify code pathways that span both the frontend (vanilla JS / React, served via Render) and the backend (Flask + PostgreSQL/SQLite REST API, hosted on Render). Each section identifies the functionality under test and provides detailed test tables with expected inputs and outputs.

1. Scope of Integration Testing

Integration tests are required for every code pathway that requires both the frontend API client (`js/api.js` → `window.API`) and the Flask backend (`backend/app.py`) to execute together. Unit tests that mock the database or isolate individual functions are out of scope here.

2. Functionality Under Test

The following frontend→backend code pathways must be covered by at least one integration test each:

2.1 Authentication

- User login with valid credentials (POST `/api/auth/login`)
- User login with invalid credentials — expect 401
- New user registration (POST `/api/auth/register`)
- Registration with duplicate email — expect 409
- Registration with missing/invalid fields — expect 400

2.2 Current User (Profile)

- Fetch current user profile (GET `/api/me`)
- Update profile fields — name, headline, location (PUT `/api/me`)
- Add education entry (POST `/api/me/education`)
- Add skill (POST `/api/me/skills`)
- Fetch profile readiness score (GET `/api/profile-readiness`)
- Fetch social state (GET `/api/me/social`)

2.3 Feed & Posts

- Fetch the post feed (GET `/api/feed`)
- Create a new post (POST `/api/feed`)
- Like a post (POST `/api/feed/:id/like`)
- Comment on a post (POST `/api/feed/:id/comments`)

- Delete a post (DELETE /api/feed/:id)

2.4 Connections & Network

- Send a connection request (POST /api/me/connections/:id)
- Accept a connection request (POST /api/me/connections/:id/accept)
- Follow/unfollow a user (POST /api/me/following/:id)
- Fetch all users for network page (GET /api/users)
- Fetch a specific user profile (GET /api/users/:id)

2.5 Messaging

- Fetch all conversations (GET /api/conversations)
- Fetch a single conversation with messages (GET /api/conversations/:id)
- Create a new conversation (POST /api/conversations)
- Send a message in a conversation (POST /api/conversations/:id/messages)

2.6 Jobs

- Fetch jobs list (GET /api/jobs)
- Fetch a single job (GET /api/jobs/:id)
- Save a job (PUT /api/me/saved-jobs/:id) — **NEW:** idempotent save operation
- Unsave a job (DELETE /api/me/saved-jobs/:id) — **NEW:** idempotent unsave operation
- Toggle save/unsave a job (POST /api/me/saved-jobs/:id) — legacy toggle endpoint
- Apply to a job (POST /api/me/applied-jobs/:id)

2.7 Notifications

- Fetch notifications (GET /api/notifications)
- Mark a single notification as read (PATCH /api/notifications/:id/read)
- Mark all notifications as read (PATCH /api/notifications/read-all)

2.8 Events & Groups

- Fetch events list (GET /api/events)
- Create a new event (POST /api/events)
- Attend an event (POST /api/events/:id/attend)
- Fetch groups list (GET /api/groups)
- Create a new group (POST /api/groups)
- Join/leave a group (POST /api/me/groups/:id/toggle)

2.9 Search

- Search with a query string (GET /api/search?q=...)

- Search with empty string — expect 200 with results or empty list

2.10 Outreach — Story #1: Message Guidance

- Generate an outreach message with valid recipient and tone (POST /api/outreach/generate)
- Generate outreach with missing recipientId — expect 400
- Generate outreach with invalid recipientId — expect 404 or 400
- Generate outreach with all optional fields populated

2.11 Outreach — Story #2: Readiness Check

- Fetch outreach readiness for current user (GET /api/outreach/readiness)
- Fetch outreach readiness for a specific userId
- Fetch readiness for non-existent user — expect 404

3. Test Tables

Each row specifies: the test ID, purpose, HTTP method and endpoint called, the inputs sent, and the expected output if the test passes. Tables are organized by functional area for clarity.

3.6 Jobs (Updated with New Endpoints)

IMPORTANT UPDATE: This section has been enhanced with new idempotent save/unsave endpoints that resolve race conditions in the original toggle-based test. The new endpoints provide deterministic state management for reliable integration testing.

Test ID	Purpose	Method / Endpoint	Inputs	Expected Output
IT-J01	Fetch all jobs	GET /api/jobs	Authorization header	HTTP 200, body contains list of job objects with id, title, and/or company fields
IT-J02	Fetch a specific job by id	GET /api/jobs/1	Authorization header	HTTP 200, body contains job object with id=1
IT-J03	Save a job (idempotent)	PUT /api/me/saved-jobs/1	Authorization header, job id=1	HTTP 200; subsequent GET /api/me/social shows job id=1 in savedJobs array. Operation is idempotent (no-op if already saved).
IT-J04	Unsave a job (idempotent)	DELETE /api/me/saved-jobs/1	Authorization header, job id=1	HTTP 200; subsequent GET /api/me/social shows job id=1 removed from savedJobs array. Operation is idempotent (no-op if not saved).
IT-J05	Deterministic save test flow	DELETE → PUT → DELETE sequence	1. DELETE to ensure unsaved state, 2. PUT to save job, 3. DELETE for cleanup	Each step verifies absolute state via GET /api/me/social. No dependency on prior test state or timing.
IT-J06	Apply to a job	POST /api/me/applied-jobs/1	Authorization header, job id=1	HTTP 200; subsequent GET /api/me/social shows job id=1 in appliedJobs array
IT-J07	Toggle save/unsave (legacy)	POST /api/me/saved-jobs/1	Authorization header, job id=1	HTTP 200; toggles saved state. Note: Deprecated in favor of PUT/DELETE for deterministic testing.

Technical Changes for Test Reliability

- **Root Cause Analysis:** The original toggle endpoint (POST) could not guarantee deterministic state due to race conditions between 'check current state' and 'toggle' operations, leading to flaky test failures.
- **Backend Implementation:** Added `save_job()` and `unsave_job()` functions to `database.py` with proper `INSERT ... ON CONFLICT DO NOTHING` and `DELETE` operations for true idempotency.
- **API Enhancements:** Introduced `PUT /api/me/saved-jobs/:id` (save) and `DELETE /api/me/saved-jobs/:id` (unsave) endpoints while maintaining POST toggle for backward compatibility.
- **Test Strategy Improvement:** IT-J03 now uses a DELETE (setup) → PUT (save) → DELETE (cleanup) sequence that guarantees known state regardless of prior test execution.

- **CI Environment Isolation:** Tests now run against local backend instances rather than shared Render deployment, eliminating cross-pipeline interference.

4. Environment Configuration

CRITICAL CI UPDATE: The continuous integration environment has been redesigned to test against local backend instances rather than the shared Render deployment. This architectural change ensures each branch tests its own code and eliminates race conditions between concurrent CI runs.

Tests are parameterized by environment variables and can run against multiple target environments:

Environment	BASE_URL	Database	Use Case
Local Development	http://localhost:5000/api	SQLite (local)	Developer testing during implementation
CI Environment	http://localhost:5000/api	SQLite (fresh per run)	Automated testing on every branch/PR
Production Testing	https://linkedin-redesign-z364.onrender.com/api	PostgreSQL (shared)	Manual verification against live deployment

CI Workflow Enhancements

- GitHub Actions now starts a local Flask server (python app.py &) before running tests
- Intelligent curl-based readiness check waits for server availability with proper error handling
- Each branch/PR tests against its own backend code instead of shared Render deployment
- Eliminated shared-state race conditions between concurrent CI pipeline runs
- Fixed curl exit code handling to prevent premature failure during server startup (|| echo fallback)

PowerShell Commands for Local Testing:

```
$env:BASE_URL = "http://localhost:5000/api" pytest backend/test_integration.py -v #
For production verification (optional): $env:BASE_URL =
"https://linkedin-redesign-z364.onrender.com/api" pytest
backend/test_integration.py -v
```

5. Test Summary & Changes

Section	Functional Area	Test Count	Status / Changes
3.1	Authentication	7	Stable
3.2	Current User / Profile	9	Stable
3.3	Feed & Posts	7	Stable
3.4	Connections & Network	6	Stable
3.5	Messaging	5	Stable
3.6	Jobs	7	✓ Enhanced: +3 idempotent endpoint tests
3.7	Notifications	3	Stable
3.8	Events & Groups	6	Stable
3.9	Search	3	Stable
3.10	Outreach — Message Guidance	4	Stable
3.11	Outreach — Readiness Check	3	Stable
TOTAL INTEGRATION TESTS		60	+3 from baseline (57)

6. Version History & Changelog

This section documents the evolution of the integration test specification and the technical improvements made to resolve flaky test behavior:

Version	Date	Changes Made	Git Commit
v1.0	Initial	Original integration test specification with 57 tests	baseline
v1.1	Fix #1	Added PUT /me/saved-jobs/:id and DELETE /me/saved-jobs/:id endpoints to replace unreliable toggle pattern	b94b0f2
v1.1	Fix #2	Rewrote IT-J03 test to use deterministic DELETE→PUT→DELETE sequence for guaranteed state control	b94b0f2
v1.2	CI Fix	Updated CI workflow to use local backend (localhost:5000) instead of shared Render deployment for isolated testing	09d48e6
v1.3	Stability	Fixed curl error handling in CI readiness check to prevent premature failures during Flask server startup	629ee53